

Design and Implementation of a Multi-Input Sorting Application

Pranav M R
2023A7PS0340G

August 27, 2025

1 Introduction

This report details the design, development and testing of a sorting application that accepts multiple input formats, processes different data types (integers, floats and strings) efficiently, and displays results through a user-friendly web interface. The application was developed as part of a course assignment for Principles of Programming Languages (CS F301).

2 Application Overview

The application is designed to sort numerical and string inputs from four different sources:

1. Keyboard input (text entry, process, and display)
2. File input (upload, process, and display)
3. Same file I/O (upload, process, and save to the same file)
4. Network input (receive data over the network, process, and display)

I developed the app following a modular approach where a single sequence of sorting logic is used for each of the different I/O mechanisms once the input is parsed correctly.

3 The Top Level

3.1 Toolchain Architecture

The application employs a web architecture with the following components:

- Backend: Used the Flask microframework for Python to handle routing and request processing.

- Frontend: Used HTML5 + CSS3 (Bootstrap 5) with Jinja2 templating for dynamic content.
- Data Processing: Used Python's built-in sorting function enhanced with chunked processing for sorting different inputs.
- Network Communication: Implemented an API using HTTP requests and JSON for fetching data from a test server.

3.2 Frontend Design

- Used the Bootstrap 5 framework for responsive design.
- Designed a tab-based interface to separate different input methods.
- Implemented a consistent results display section for all input types.
- Loading indicators and alerts for user feedback (in case of errors and success messages).

4 The Base

4.1 Programming Languages and Components

4.1.1 Main Sorting Implementation

Python was selected for the core sorting functionality due to its:

- Built-in efficient sorting algorithm (Timsort: a stable sorting algorithm derived from merge sort and insertion sort).
- Excellent memory management for large datasets.
- Easy integration with web frameworks.

I split the input data into manageable chunks, sort each chunk individually, and then merge the sorted chunks. This approach ensures efficient memory usage and maintains good performance even with very large input sets.

4.1.2 Input/Output Handling

- **Keyboard Input:** Processes text data using string parsing and conversion. The output is displayed directly on the webpage.
- **File Input:** Handles file uploads, reads content, and processes the data. The sorted output is displayed on the webpage.
- **Same File I/O:** Reads from and writes to the same file, using the same sorting logic. The file is updated with sorted content.
- **Network Input:** Receives data from an API endpoint, using a client-server architecture. The sorted output is displayed on the webpage.

4.2 Pre/Post Processing

- **Pre-processing:** Input validation, error handling, and type conversion (if the data is not of string type).
- **Memory Management:** Chunked processing for large files to avoid memory exhaustion.
- **Post-processing:** Formatting of sorted results for display.

5 The Manufacturing

5.1 IDE and Development Tools

I used Visual Studio Code as the primary development environment since I am comfortable with it. It also support Python and web development well with extensions like Python and Live Server.

5.2 Development Process

1. Implemented the core functionality (sorting algorithm and chunk processing logic).
2. Developed individual input methods (keyboard, file, same file I/O, network) and parsing logic.
3. Designed and implemented the frontend interface using Bootstrap.
4. Integrated the frontend with the backend using Flask routes and Jinja2 templates.

5.3 Testing Process

- First, I tested the sorting algorithm with simple inputs for various data types through the terminal.
- After developing each input method, I tested them individually with both small and large inputs.
- Tested network input using a mock server to simulate data fetching (server logic in `server.py`).
- Tested the complete application with various input types and sizes, including edge cases (included in the `data` folder).